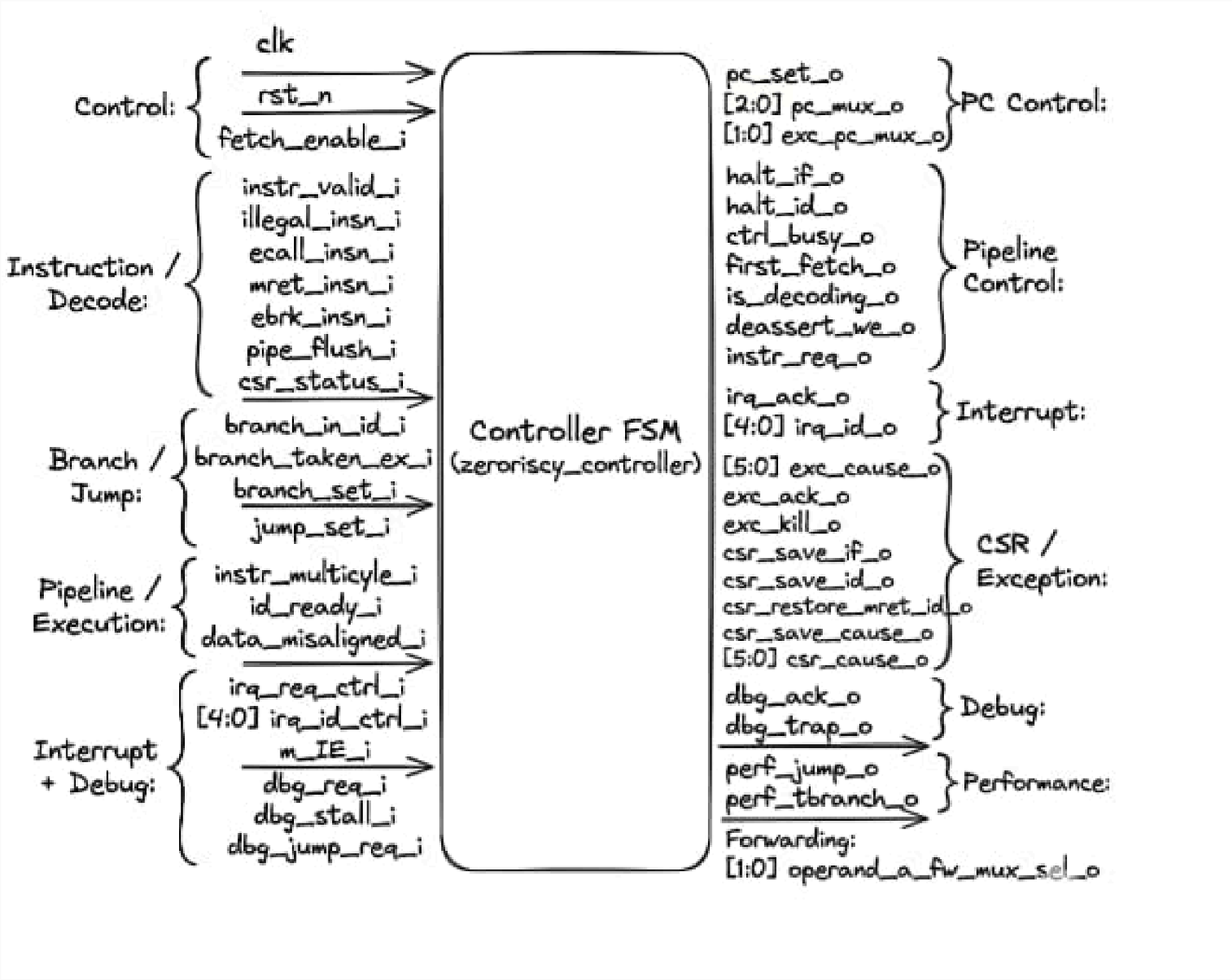
Zero Riscy Controller

Overview

- The Concept: The "Central Dispatcher."
- Key Function: It does not process data; it manages the state machine.
- Core Responsibilities:
  - Synchronizing pipeline stages.
  - Handling exceptions and interrupts.
  - Managing stalls and data hazards.



Instruction Fetch (IF)

- The Goal: Getting instructions from memory to the CPU.

- Input Signal:
  - instr_valid_i: The "I’m ready" signal from the prefetcher.

- Output Signals:
  - instr_req_o: The master "On" switch for memory requests.
  - pc_set_o: The "Redirect" command to override the program counter.
  - pc_mux_o: Selector for the jump destination (Jump vs. Exception vs. Boot).
  - halt_if_o: The emergency brake to stop all fetching.

Instruction Decoder (ID)

- The Goal: Translating raw bits into actionable CPU commands.

- Input Signals:
- Instruction Types: jump_set_i, ecall_insn_i, illegal_insn_i.
- id_ready_i: The "Green Light" indicating the ID stage is finished.

- Output Signals:
- is_decoding_o: Formal permission to start the decoding process.
- halt_id_o: Holds the current instruction in place (used during multi-cycle ops).
- deassert_we_o: The Safety Net. Disables writing to registers if an error is detected.

Interrupt & Debug Units

- Interrupt Signals (Logic):
    irq_req_ctrl_i (In): External hardware needs attention.
    irq_ack_o (Out): Acknowledging the stop of the current program.

- Debug Signals (Human Interface):
    dbg_req_i (In): The human wants to pause the CPU via GDB.
    dbg_ack_o (Out): Confirmation that the pipeline is clear and the CPU is paused.

Load-Store Unit (LSU)

- The Goal: Managing data movement to/from RAM.
- The Challenge: Misaligned Data.
- Key Signal Flow:

data_misaligned_i (In): The LSU detects an address that doesn't line up on 4 -byte boundaries.

operand_a_fw_mux_sel_o (Out): The Controller triggers a "Shortcut" (Forwarding) to re-route and fix the data alignment before it enters the Execution unit.

Prefetching

The prefetcher is like a scout. It stays a few steps ahead of the CPU, grabbing instructions from memory and putting them in a small queue so the CPU never has to wait.

- instr_req_o (Controller → Prefetcher): This is the master "On" switch. If the Controller is in a SLEEP or RESET state, it turns this off to save power.

- instr_valid_i (Prefetcher → Controller): The prefetcher says, "I have an instruction ready in my buffer for you to decode."

- pc_set_o & pc_mux_o (Controller → Prefetcher): These are the "Redirect" signals.
  - Normally, the prefetcher just does PC + 4.
  - But if the Controller sees a Jump or Branch, it sends pc_set_o = 1. This tells the prefetcher: "Throw away what you're currently fetching; we are teleporting to the address specified by pc_mux_o."

Stall signals (Brakes)

A stall happens when the CPU needs to "pause" a stage because the next stage isn't ready or data isn't available yet.

- halt_if_o (Controller → IF Stage): "Stop fetching." We use this during a SLEEP state or if the Debugger has paused the chip.

- halt_id_o (Controller → ID Stage): "Freeze the current instruction." This happens if an instruction takes multiple cycles (like a Divide) or if we are waiting for an Interrupt to be processed.

- id_ready_i (ID Stage → Controller): This is the "Green Light." The Decode stage tells the Controller, "I am finished with this instruction; you can move the pipeline forward now."

Forwarding Signals (The "Shortcuts")

Forwarding is a trick to avoid stalls. Normally, an instruction writes its result to a register, and the next instruction reads it. But writing takes time. Forwarding "grabs" the result as soon as it's calculated and feeds it directly into the next instruction.

- operand_a_fw_mux_sel_o (Controller → ID Stage): This tells the ID stage where to get its data from.
  - In your specific instantiation, this is tied to data_misaligned_i.
  - The Meaning: If data is "misaligned" (not sitting perfectly on a 4-byte boundary), the Controller detects this and switches the Mux. Instead of reading the "wrong" data from the register file, it routes the corrected data from the Load-Store Unit back into the execution unit.

Performance & Status Signals

These help the outside world (and developers) see how well the "traffic control" is working.

- perf_jump_o & perf_tbranch_o: The Controller sends a pulse here every time a Jump or a Branch is successfully taken. These are connected to "Performance Counters" to calculate things like "Instructions Per Cycle" (IPC).

- ctrl_busy_o: This is the "Status Light." It stays high as long as the Controller is doing something. If the CPU hits a WFI (Wait For Interrupt) instruction and goes to SLEEP, this goes low, telling the system the CPU is idling.

Summary

Block	Controller Listens For...	Controller Commands...
Fetch Unit	Is instruction valid?	Fetch next, Jump, or Halt.
Decoder	Is it a Jump? Is it Illegal?	Start decoding, Stop, or Clear errors.
Interrupts	Is there an emergency?	Acknowledge and switch to handler.
Debug	Does the user want to stop?	Confirm pause and report status.

Summary

Signal Type	Primary Job	Key Consequence
Prefetch	Speed up instruction delivery.	Prevents the CPU from waiting on slow Memory.
Stalls	Stop the pipeline.	Prevents "Hazard" errors when data isn't ready.
Forwarding	Create "Shortcuts" for data.	Increases speed by reducing the need for Stalls.